

MF-GARS: Multi-Fidelity Gap-Aware Ranking Search for Heterogeneous In-Sensor Vision-Processing Systems

Zecheng Xuan¹, Zheng Lan², Jianhua Zhang¹, Xiupeng Shi^{1*}, Yanan Liu^{1*}, *Member, IEEE*

Abstract—Computational Pixel Matrix (CPM)—field-programmable gate array (FPGA) systems reduce per-frame data movement for edge video analysis, but physical CPM readout can change backend representations and invalidate simulation-only rankings. We present Multi-Fidelity Gap-Aware Ranking Search (MF-GARS) for ranking jointly trained CPM-frontend and FPGA-backend candidates from limited physical measurements. Its multi-task mixture-of-experts model uses simulation accuracy as a prior to predict the sim-to-real accuracy gap, latency, and FPGA resource usage. A normalized reward combines calibrated accuracy, bytes per frame, latency, and resource utilization; a Top- K stability-aware policy allocates measurements. We physically evaluate 320 of 3,000 configurations as a frozen physical oracle benchmark and reserve 2,680 disjoint candidates for deployment ranking. With 20 visible Street View House Numbers records, MF-GARS obtains 68.0% Hit@10 versus 62.2% for Gap-XGB, although its MAE advantage is not significant. In budget-20 masked-record replay, MF-GARS achieves 61.45% Mean@10 true reward, 3.24 percentage points above Direct-Greedy, and ties the best Best@10 reward at 71.81%. A controlled comparison then gives all rankers the same MF-GARS-acquired 40-record calibration set. Yale Face results reproduce the finite-budget shortlist advantage. MF-GARS therefore supports measurement-efficient CPM-FPGA configuration ranking for per-frame edge video processing.

Index Terms—Automated machine learning, computational pixel matrices, edge video analytics, FPGA acceleration, in-sensor computing, deployment-aware candidate selection

I. INTRODUCTION

EDGE video analysis demands low per-frame latency, bounded link bandwidth, and efficient use of constrained hardware. In-sensor processors, including computational pixel matrices (CPMs), move sensing and feature extraction onto the focal plane. Liu et al. demonstrated lightweight CNN inference with strided convolutions on a pixel processor array [1]. A binarized on-sensor CNN was later used for localization and coarse segmentation [2]. Dynamic model swapping exposed the storage and operator limits of pixel processor arrays [3], while a later survey organized their visual-perception architectures [4]. Dudek et al. demonstrated high-rate sensor-level vision for agile robots [5]. FPGAs complement CPMs with programmable, parallel CNN backends: layer-specific mixed-precision dataflows reduce implementation cost [6], FPGA deconvolution accelerators improve arithmetic efficiency [7], and real-time FPGA detectors address accuracy–latency trade-offs [8]. For a stream rate of f_v frames/s, a representation of M bytes/frame requires a payload bitrate of $8f_vM$

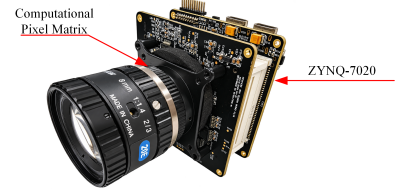


Fig. 1. CPM-FPGA heterogeneous hardware platform.

bits/s. We therefore optimize per-frame accuracy, transferred bytes, FPGA latency, and LUT/FF/DSP/BRAM utilization; workload-dependent power and energy remain outside the measured objective.

Despite these complementary roles, jointly optimizing the coupled system remains difficult. Each sensor-side representation changes the backend input distribution, communication volume, and FPGA implementation requirements. Simulation is inexpensive but incompletely captures physical readout, whereas candidate-level hardware measurements are costly. Consequently, configurations that appear attractive in simulation may rank differently after physical readout and FPGA implementation.

The central challenge is therefore to rank many jointly trained frontend-template/backend candidates from inexpensive simulation descriptors and scarce physical measurements. Candidates sharing a frontend structural template still have separately learned sensor-side parameters and produce distinct physical representations. Measuring one candidate therefore does not label another. A practical method must select complete candidates for measurement and rank the remaining inventory under communication, latency, and FPGA-capacity constraints.

Multi-Fidelity Gap-Aware Ranking Search (MF-GARS) addresses this coupled-ranking problem by treating simulation accuracy as a low-cost prior. Selectively acquired real-CPM accuracy provides the high-fidelity label. A multi-task mixture-of-experts (MoE) surrogate predicts residual accuracy degradation and FPGA costs. A hardware-aware reward and Top- K policy then support constrained ranking and finite-budget replay.

The main contributions of this paper are threefold:

- *Coupled physical benchmark.* We construct a heterogeneous CPM-FPGA benchmark in which each discrete frontend-template/backend pairing is trained and evaluated as a distinct candidate. The inventory contains 3,000 candidates, including 320 complete physical records for a frozen oracle benchmark and a disjoint 2,680-candidate deployment pool. Across ran-

¹School of Microelectronics, Shanghai University, Shanghai, China.

²XXX Institute, XXX University, City, China.

*Corresponding author: yanan_liu@shu.edu.cn.

dom, frontend-grouped, and backend-grouped validation, the full oracle reference reaches $R^2 = 94.83\%$, 94.71% , and 93.05% , respectively. We evaluate this protocol on SVHN [9] and Yale Face [10].

- *Data-efficient multi-task sim-to-real surrogate.* We formulate coupled deployment as multi-fidelity AutoML. A multi-task MoE predicts the sim-to-real accuracy gap and candidate-level FPGA costs while retaining candidate-specific effects from jointly trained CPM and backend parameters. With 20 visible SVHN records, it gives MAE = 4.19 pp, Hit@10 = 68.0%, and Mean@10 true reward = 74.03%, compared with MAE = 4.32 pp and Hit@10 = 62.2% for Gap-XGB; the MAE difference is not significant ($p = 0.0708$).

- *Top- K stability-aware active screening.* We combine bootstrap Top- K membership, boundary uncertainty, and structural-template/backend coverage to allocate scarce real-CPM measurements. In budget-20 end-to-end replay, MF-GARS reaches SVHN Mean@10 true reward = 61.45% (+3.24 percentage points over Direct-Greedy) and Yale Face Mean@10 true reward = 61.02% (+2.85 percentage points). Separate end-to-end acquisition and shared-calibration protocols distinguish measurement-policy quality from final-ranking quality while ranking the complete deployment pool.

II. RELATED WORK

We review four foundations of coupled CPM-FPGA candidate ranking: in-sensor computing, FPGA deployment, task-oriented compression, and constrained AutoML.

A. In-Sensor Computing and Computational Pixel Matrices

In-sensor computing reduces data movement by integrating sensing, storage, and computation. Carey et al. reported a 100,000-fps SIMD vision sensor with an embedded pixel processor array [11]. Dudek et al. showed that sensor-level processing supports agile-robot perception [5]. Bose et al. demonstrated CNN inference directly on pixel processor arrays [12], and subsequently embedded fast convolutional networks in the same class of processors [13]. Liu et al. used strided convolutions for lightweight sensor-side CNN inference [1]; a separate study used binarization for localization and segmentation [2]. Dynamic model swapping further characterized register, operator, and storage limits [3]. Liu et al. surveyed the broader in-sensor visual-perception design space [4]. So et al. introduced PixelRNN to learn recurrent in-pixel representations for end-to-end perception [14]. These studies establish the communication and latency motivation for CPMs, but they do not rank a large inventory of jointly trained CPM-FPGA candidates from sparse physical measurements.

Most prior CPM studies optimize a fixed sensor program or treat sensor output as the endpoint. They do not jointly rank multiple physical frontend representations against a large backend library. Nor do they use sparse physical measurements to correct simulation-based rankings. MF-GARS instead treats each jointly trained CPM-template/backend instance as one candidate and learns the residual between simulated and physical CPM accuracy.

B. FPGA Backend Implementation in Heterogeneous Edge Vision

FPGA vision efficiency depends on quantization, dataflow, memory organization, and resource allocation. Nguyen et al. used layer-specific mixed-precision dataflows to reduce FPGA CNN cost [6]. Chang et al. jointly optimized an FPGA deconvolution accelerator for super-resolution [7]. Li et al. targeted high-accuracy real-time detection on FPGA [8]. Jia et al. developed an FPGA-accelerated 4K neural video-coding system [15], whereas Zhang et al. studied CPU-FPGA co-design for video analysis [16]. These works characterize implementation trade-offs but do not model the physical CPM representation as part of the candidate identity.

These accelerators nevertheless usually receive a fixed digital tensor. Here, changing the CPM frontend alters output shape, byte count, representation statistics, buffering demand, and backend performance. A backend efficient for one frontend may therefore be unsuitable for another. MF-GARS consequently assigns latency and LUT/FF/DSP/BRAM costs to the coupled candidate and predicts them jointly with the physical accuracy gap.

C. Task-Oriented Compression for Machine Vision

Task-oriented compression replaces reconstruction-only coding with learned rate-distortion optimization and semantic representations. Sun et al. formulated semantic structured image coding for machine tasks [17]. Torfason et al. showed that compressed features can support image understanding without decoding [18]. Hu et al. proposed scalable coding for human and machine vision [19]. Li et al. studied reinforcement-learning task-driven coding [20], and Yan et al. proposed semantics-to-signal scalable coding [21]. Kim et al. developed learnable multi-scale feature compression for video coding for machines [22]. Zhang et al. investigated cross-modality transfer for scalable semantic compression [23]. Lu et al. moved task-oriented preprocessing ahead of a standard codec [24]. SACoD extended sensor-algorithm co-design to a lensless frontend and CNN [25]. These methods optimize communication and task accuracy, but they generally assume digital or differentiable inputs rather than physical CPM readout.

Most task-oriented codecs still assume digital or differentiable sensing. They do not account for analog-operation mismatch, finite-precision CPM readout, or downstream FPGA feasibility. They also optimize average task performance rather than the stability of a compact deployment shortlist. MF-GARS uses measured CPM output as the high-fidelity target, includes communication and FPGA costs in its reward, and allocates measurements according to Top- K stability.

D. AutoML and Constrained Deployment Optimization

Auto-WEKA combines algorithm selection with hyperparameter optimization [26]. Auto-sklearn adds robust automated pipeline construction [27]. These methods provide software-model selection, but their default objectives and spaces do not represent a physical sensor-accelerator pair. A CPM-FPGA candidate combines frontend operators, backend architecture,

quantization, mapping, communication, and physical-readout effects.

SMAC3 uses random-forest Bayesian optimization [28], and OpenBox generalizes black-box optimization to reusable service interfaces [29]. Their fidelity variables usually represent data, iterations, or computation, not discrepancies between deterministic simulation and physical CPM readout. MF-GARS retains the finite-budget perspective while defining simulation and physical readout as low- and high-fidelity observations.

HyperMapper supports categorical, multi-objective, feasibility-aware design-space exploration [30]. AutoDSE automates high-level-synthesis exploration for FPGA accelerators [31]. Both address hardware constraints, but neither models a sensor whose physical output changes the task label or allocates measurements to stabilize a deployment Top- K . MF-GARS addresses these gaps with sim-to-real residual learning, shared hardware-prediction heads, and stability-aware acquisition for coupled sensor-processor candidates.

III. METHODOLOGY

We formulate deployment as multi-fidelity AutoML over a complete inventory of coupled CPM-FPGA configurations. The space includes CPM sensing operators, representation shapes, backend instances, quantization policies, and FPGA mappings. The frontend determines compression, communication, and the backend input distribution. The backend determines inference accuracy, latency, and resource use. Physical readout executes the frontend on sensor signals and exports its representation through the digital interface. Analog-operation mismatch, finite-precision conversion, and readout variability distinguish this output from deterministic simulation. We therefore treat simulation and physical measurement as separate fidelity levels. A stratified 320-configuration physical oracle benchmark supplies high-fidelity labels for controlled evaluation. Final deployment ranking uses the disjoint pool of 2,680 unmeasured configurations.

A. CPM-FPGA Hardware Platform

The platform couples a CPM frontend with an FPGA backend (Fig. 1). The CPM performs analog signal processing, sensor-level data compression, 8-bit conversion, and feature readout (Fig. 2). AXI infrastructure transfers the stream to an HLS CNN accelerator, while the processing system manages control. A uniform FPGA implementation-report protocol supplies latency and LUT/FF/DSP/BRAM labels for surrogate supervision and feasibility screening. The objective therefore spans the complete physical readout-to-inference path, unlike software-only AutoML.

B. Overview of the MF-GARS Pipeline

Fig. 3 summarizes MF-GARS. A joint descriptor encodes frontend compression, output shape, backend mapping, and simulation accuracy. The multi-task MoE predicts the sim-to-real gap and FPGA costs. Calibrated accuracy and cost then guide Top- K screening or the next real-CPM query.

Let $\mathbf{x}_0 \in \mathbb{R}^{H_{\text{in}} \times W_{\text{in}} \times C_{\text{in}}}$ denote the raw sensor input. The current setup uses a $256 \times 256 \times 1$ grayscale frame. For candidate c , the CPM function combines structural template s_f with candidate-specific learned parameters $\theta_{\text{CPM}}^{(c)}$. It emits a compressed representation $\mathbf{z}_c \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}}$. Candidate-specific backend parameters $\theta_{\text{BE}}^{(c)}$ then map this representation to a prediction. The dataflow is

$$\begin{aligned} \mathbf{z}_c &= F_{\text{CPM}}(\mathbf{x}_0; s_f, \theta_{\text{CPM}}^{(c)}), \\ \hat{y}_c &= F_{\text{BE}}(\mathbf{z}_c; b, \theta_{\text{BE}}^{(c)}). \end{aligned} \quad (1)$$

Software simulation supplies low-cost screening features, whereas real-CPM readout supplies high-fidelity labels. Joint selection is necessary because the frontend changes information content, shape, and readout cost. It can therefore change the preferred backend. Even when two candidates share s_f , separately learned $\theta_{\text{CPM}}^{(c)}$ produce candidate-specific representations. A measured output or label from one candidate cannot serve as the physical observation of another.

C. CPM Frontend Compression Space

The frontend space contains only CPM-compatible sensing and readout operators for final deployment. Each structural template is represented as

$$s_f = (o_f, H_{\text{out}}, W_{\text{out}}, C_{\text{out}}, q, t), \quad (2)$$

where o_f is the ordered operator sequence, $(H_{\text{out}}, W_{\text{out}}, C_{\text{out}})$ is the output shape, q is the byte width, and t is the readout format. Here, $q = 1$ byte because the deployed frontend uses 8-bit readout. The six s_f values denote structural templates rather than fixed-weight functions. Each coupled candidate is trained separately and has its own $\theta_{\text{CPM}}^{(c)}$.

The compression ratio is defined by the raw-input size divided by the frontend output size:

$$\rho(s_f) = \frac{H_{\text{in}} W_{\text{in}} C_{\text{in}}}{H_{\text{out}} W_{\text{out}} C_{\text{out}}}. \quad (3)$$

The communication cost from the CPM to the FPGA is measured in bytes per frame as

$$M(s_f) = H_{\text{out}} W_{\text{out}} C_{\text{out}} q. \quad (4)$$

For the current $256 \times 256 \times 1$ input, the raw-frame transfer cost is $M_{\text{raw}} = 256 \times 256 \times 1 = 65536$ bytes. At a stream rate of f_v frames/s, $M(s_f)$ corresponds to a payload bitrate of $8f_v M(s_f)$ bits/s before protocol overhead. We report bytes/frame because the experiments vary the coupled configuration rather than the temporal sampling rate. Because $q = 1$ and the raw input size is fixed, $\rho(s_f)$ and $M(s_f)$ deterministically encode the same communication property. The deployment reward therefore uses $M(s_f)$ without a separate compression-ratio term.

Only operators executable on the CPM/readout path are valid deployment candidates. Candidates can share a compression ratio while differing in output shape and downstream behavior.

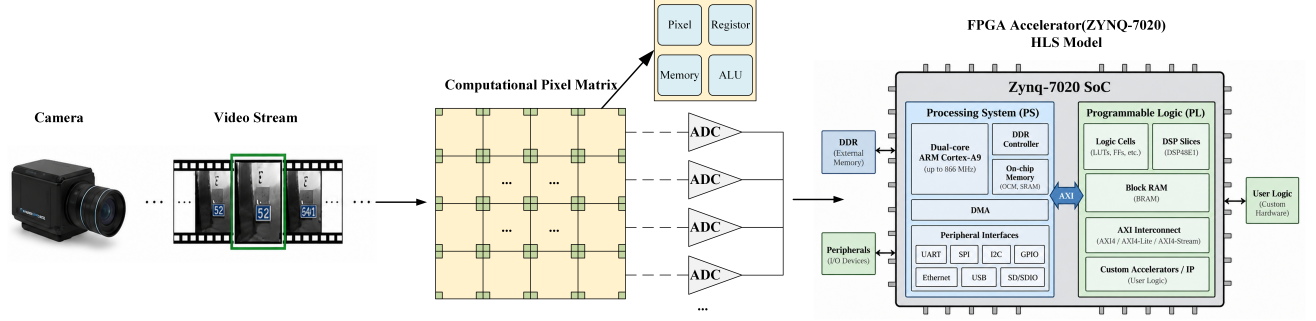


Fig. 2. Hardware path from CPM sensing, processing, and readout to the FPGA HLS accelerator.

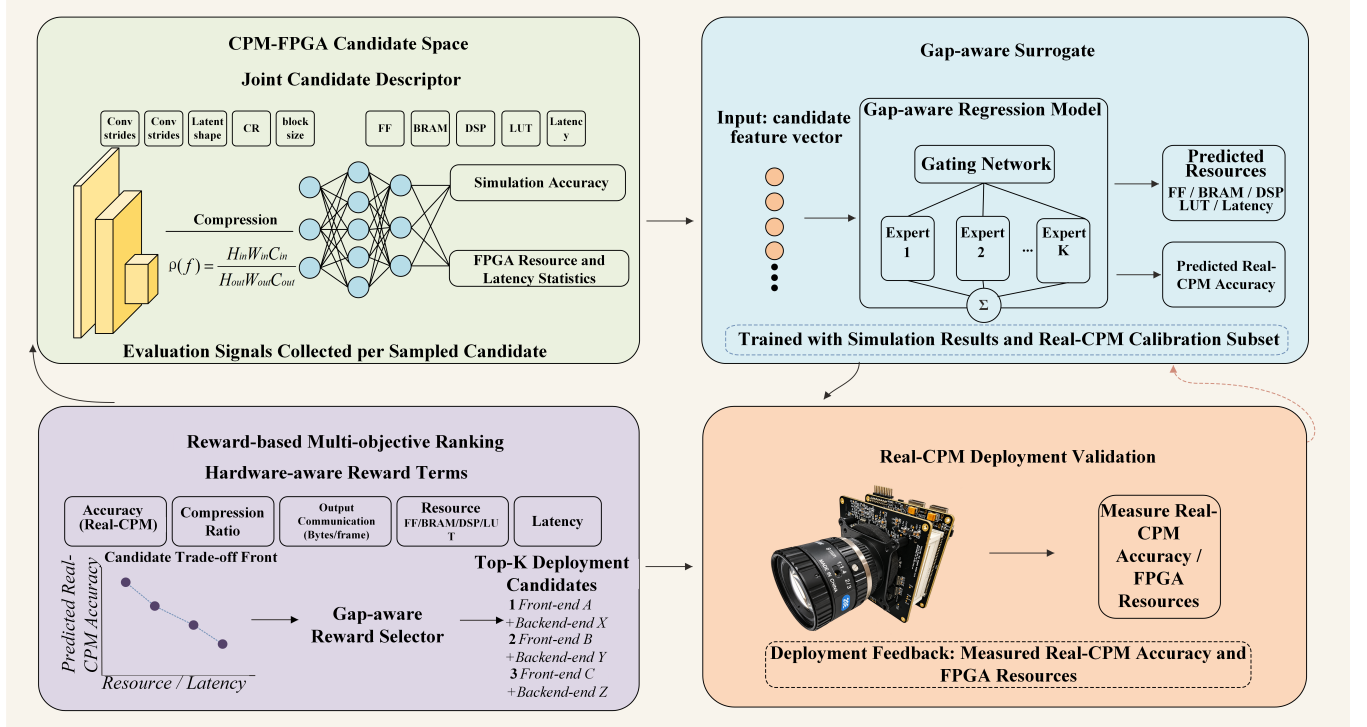


Fig. 3. MF-GARS workflow from coupled descriptors and multi-fidelity supervision to calibrated Top- K screening.

D. FPGA Backend Resource Space

Each backend candidate is represented as

$$b = (a, w, m), \quad (5)$$

where a denotes the CNN architecture, w denotes the weight/activation precision policy, and m denotes the FPGA mapping choice. We also record pure backend accuracy $A_{\text{pure}}(b)$ on the original digital input without a CPM frontend. This frontend-free digital-input reference is not a guaranteed upper bound and does not measure robustness to CPM-conditioned inputs.

Because frontend output shape changes buffering, data movement, and accelerator workload, hardware cost belongs to the coupled candidate rather than b alone. We collect the candidate-level FPGA resource vector

$$\mathbf{r}(c) = [r_{\text{LUT}}(c), r_{\text{FF}}(c), r_{\text{DSP}}(c), r_{\text{BRAM}}(c)], \quad (6)$$

and the corresponding latency $L(c)$. Every configuration in the 320-configuration physical oracle benchmark has a complete implementation record. Its latency and LUT/FF/DSP/BRAM labels follow the same reporting protocol. For configurations outside this benchmark, the hardware heads predict these quantities during screening. These heads provide common feasibility estimates for configurations without implementation reports; they do not replace candidate-level physical evaluation. Resource-aware ranking uses the predicted or measured unified normalized hardware score in Eq. (20). Its weights are ω_{LUT} , ω_{FF} , ω_{DSP} , and ω_{BRAM} . Pure backend accuracy is insufficient because it ignores the frontend-induced representation shift. The backend must therefore be evaluated jointly with the CPM output.

The backend library contains 500 configurations with distinct accuracy, latency, and resource-consumption profiles. MF-GARS ranks coupled candidates from this library rather

than generating architectures. Pairing these variants with six CPM structural templates yields the complete 3,000-configuration inventory. This inventory exposes accuracy-latency-resource trade-offs under a fixed FPGA budget.

E. Coupled Candidate Formulation

The searchable deployment unit is a jointly trained coupled candidate

$$\begin{aligned} c &= (s_f, b, \Theta_c), \quad s_f \in \mathcal{S}_F, \quad b \in \mathcal{B}, \\ \Theta_c &= \{\theta_{\text{CPM}}^{(c)}, \theta_{\text{BE}}^{(c)}\}. \end{aligned} \quad (7)$$

Its discrete identity is the pair (s_f, b) . Candidate-specific learned parameters distinguish its physical representation and accuracy without expanding the discrete inventory. The full coupled candidate inventory is therefore

$$\begin{aligned} \mathcal{C}_{\text{all}} &= \mathcal{S}_F \times \mathcal{B}, \\ |\mathcal{S}_F| &= 6, \quad |\mathcal{B}| = 500, \quad |\mathcal{C}_{\text{all}}| = 3000, \\ \mathcal{C}_{\text{oracle}} &\subset \mathcal{C}_{\text{all}}, \quad |\mathcal{C}_{\text{oracle}}| = 320, \\ \mathcal{C}_{\text{deploy}} &= \mathcal{C}_{\text{all}} \setminus \mathcal{C}_{\text{oracle}}, \quad |\mathcal{C}_{\text{deploy}}| = 2680. \end{aligned} \quad (8)$$

The 2,680 configurations in $\mathcal{C}_{\text{deploy}}$ form the complete deployment pool. Every method scores all 2,680 candidate IDs under the same hardware-constraint rule. No method-specific subpool is formed before ranking. The constraint rule is fixed before method-specific reward scoring; real-CPM accuracy is never available for candidates in $\mathcal{C}_{\text{deploy}}$ during ranking. Each candidate has simulation accuracy $A_{\text{sim}}(c)$, measured real-CPM accuracy $A_{\text{real}}(c)$, and communication cost $M(c)$. Hardware reports provide $L(c)$ and $\mathbf{r}(c)$ for implemented candidates. Otherwise, the surrogate supplies $\hat{L}(c)$ and $\hat{\mathbf{r}}(c)$. The platform budgets are

$$\mathbf{B} = [B_{\text{LUT}}, B_{\text{FF}}, B_{\text{DSP}}, B_{\text{BRAM}}], \quad (9)$$

with latency budget L_{max} . For final deployment, the method-independent feasibility indicator is

$$I_{\text{feas}}(c) = \mathbf{1}[\hat{\mathbf{r}}_{\text{com}}(c) \preceq \mathbf{B} \wedge \hat{L}_{\text{com}}(c) \leq L_{\text{max}}]. \quad (10)$$

The common hardware estimates are fitted from shared visible calibration records and reused by every final-ranking method. Feasibility treatment is therefore method independent. We implement the constraint within the ranking score,

$$\hat{\mathcal{R}}_{\text{con}}(c) = \begin{cases} \hat{\mathcal{R}}(c), & I_{\text{feas}}(c) = 1, \\ -\infty, & I_{\text{feas}}(c) = 0, \end{cases}$$

so every method performs one constrained Top-60 selection over the complete 2,680-candidate deployment pool. During oracle replay, the same rule uses hardware information visible at each step. Measured reports replace predictions after a candidate is queried.

$$c^* = \arg \max_{c \in \mathcal{C}_{\text{deploy}}} \hat{\mathcal{R}}_{\text{con}}(c), \quad (11)$$

where $\hat{\mathcal{R}}(c)$ is the predicted normalized deployment reward defined below. The constrained score $\hat{\mathcal{R}}_{\text{con}}(c)$ applies the fixed hardware rule to this prediction. Because obtaining $A_{\text{real}}(c)$ for every candidate is costly, ranking must be calibrated from a limited real-CPM subset.

F. Multi-task MoE Surrogate for Gap and Hardware Prediction

Real-CPM calibration executes each selected candidate-specific frontend with 8-bit ADC/DREG readout. Oracle configurations $\mathcal{C}_{\text{oracle}}$ use image set D_{cal} , which is disjoint from held-out test set D_{test} .

To bridge software simulation and real deployment, we define the sim-to-real gap as

$$\Delta(c) = A_{\text{sim}}(c) - A_{\text{real}}(c). \quad (12)$$

The multi-task MoE jointly predicts this residual and FPGA indicators from

$$\mathbf{x}(c) = \begin{bmatrix} A_{\text{sim}}(c), \rho(s_f), H_{\text{out}}, W_{\text{out}}, C_{\text{out}}, M(s_f), \\ A_{\text{pure}}(b), \psi(b) \end{bmatrix}^{\top}, \quad (13)$$

where $\psi(b)$ contains backend architecture, quantization, mapping, and reuse-factor descriptors. Hardware labels are prediction targets rather than inputs.

Let $\mathcal{T}_{\text{hw}} = \{L, \text{LUT}, \text{FF}, \text{DSP}, \text{BRAM}\}$ denote the hardware targets. For each output task $m \in \{\Delta\} \cup \mathcal{T}_{\text{hw}}$, the MoE prediction is

$$\hat{y}_m(c) = \sum_{i=1}^E g_i(\mathbf{x}(c)) h_{m,i}(\mathbf{x}(c)), \quad (14)$$

where $g_i(\cdot)$ is the gating weight for expert i , and $h_{m,i}(\cdot)$ is its task-specific output for target m . The full multi-task output is

$$\hat{\mathbf{y}}(c) = \begin{bmatrix} \hat{\Delta}(c), \hat{L}(c), \hat{r}_{\text{LUT}}(c), \hat{r}_{\text{FF}}(c), \\ \hat{r}_{\text{DSP}}(c), \hat{r}_{\text{BRAM}}(c) \end{bmatrix}. \quad (15)$$

All candidates in $\mathcal{C}_{\text{oracle}}$ have complete gap, latency, LUT, FF, DSP, and BRAM supervision in the offline benchmark. Each experiment exposes only its budget-permitted subset $\mathcal{C}_{\text{train}} \subseteq \mathcal{C}_{\text{oracle}}$. The surrogate uses the normalized multi-task loss

$$\mathcal{L}_{\text{MoE}} = \lambda_{\Delta} \mathcal{L}_{\Delta} + \sum_{m \in \mathcal{T}_{\text{hw}}} \lambda_m \mathcal{L}_m, \quad (16)$$

where

$$\mathcal{L}_{\Delta} = \frac{1}{|\mathcal{C}_{\text{train}}|} \sum_{c \in \mathcal{C}_{\text{train}}} (\Delta(c) - \hat{\Delta}(c))^2 \quad (17)$$

and

$$\mathcal{L}_m = \frac{1}{|\mathcal{C}_{\text{train}}|} \sum_{c \in \mathcal{C}_{\text{train}}} (y_m(c) - \hat{y}_m(c))^2. \quad (18)$$

Targets are normalized. Within each outer candidate-level fold, hyperparameters are selected using only the training candidates. No real-CPM or hardware label from an outer held-out candidate enters hyperparameter selection or model fitting.

The surrogate gap head gives calibrated real accuracy as

$$\hat{A}_{\text{real}}(c) = A_{\text{sim}}(c) - \hat{\Delta}(c). \quad (19)$$

Candidate ranking uses this calibrated quantity instead of simulation accuracy alone.

G. Multi-fidelity Reward and Active Ranking

At iteration t , $\mathcal{M}_t$ and $\mathcal{U}_t$ denote the measured and unmeasured oracle sets. One candidate-level measurement obtains $A_{\text{real}}(c)$, $L(c)$, and the complete LUT/FF/DSP/BRAM vector $\mathbf{r}(c)$ for the selected candidate. The already available $A_{\text{sim}}(c)$ then determines $\Delta(c)$. Simulation accuracy, output bytes, and descriptors are available beforehand and do not consume the real-CPM budget. Every coupled candidate has separately learned $\theta_{\text{CPM}}^{(c)}$ and $\theta_{\text{BE}}^{(c)}$. Its physical representation and measured labels therefore cannot transfer to another candidate, even one sharing s_f .

We distinguish the reward predicted during search from the true reward used for evaluation. For an unmeasured candidate, the normalized predicted terms are

$$\begin{aligned}\tilde{A}^{\text{pred}}(c) &= \text{clip}\left(\hat{A}_{\text{real}}(c), 0, 1\right), \\ \tilde{M}(c) &= \text{clip}\left(\frac{M(c)}{M_{\text{raw}}}, 0, 1\right), \\ \tilde{L}^{\text{pred}}(c) &= \text{clip}\left(\frac{\hat{L}(c)}{L_{\text{max}}}, 0, 1\right), \\ \tilde{R}_{\text{hw}}^{\text{pred}}(c) &= \omega_{\text{LUT}} \text{clip}\left(\frac{\hat{r}_{\text{LUT}}(c)}{B_{\text{LUT}}}, 0, 1\right) \\ &\quad + \omega_{\text{FF}} \text{clip}\left(\frac{\hat{r}_{\text{FF}}(c)}{B_{\text{FF}}}, 0, 1\right) \\ &\quad + \omega_{\text{DSP}} \text{clip}\left(\frac{\hat{r}_{\text{DSP}}(c)}{B_{\text{DSP}}}, 0, 1\right) \\ &\quad + \omega_{\text{BRAM}} \text{clip}\left(\frac{\hat{r}_{\text{BRAM}}(c)}{B_{\text{BRAM}}}, 0, 1\right),\end{aligned}\quad (20)$$

where the resource weights sum to one. The predicted raw reward is

$$\hat{\mathcal{R}}_{\text{raw}}(c) = \tilde{A}^{\text{pred}}(c) - \lambda_M \tilde{M}(c) - \lambda_L \tilde{L}^{\text{pred}}(c) - \lambda_R \tilde{R}_{\text{hw}}^{\text{pred}}(c). \quad (21)$$

MF-GARS ranks with the normalized predicted reward

$$\hat{\mathcal{R}}(c) = \frac{\hat{\mathcal{R}}_{\text{raw}}(c) - R_{\min}}{R_{\max} - R_{\min}}, \quad (22)$$

where $R_{\min} = -(\lambda_M + \lambda_L + \lambda_R)$ and $R_{\max} = 1$ follow from the clipped terms. For evaluation, the same equations define $\mathcal{R}^{\text{true}}(c)$. Measured $A_{\text{real}}(c)$, $L(c)$, and $\mathbf{r}(c)$ replace their predicted counterparts. Thus, $\hat{\mathcal{R}}$ is available during ranking, whereas $\mathcal{R}^{\text{true}}$ is an oracle or post-selection quantity. The latter remains unavailable for unmeasured deployment candidates. All reported Reward (%), Mean@10 Reward, Best@10 Reward, and regret values use $100\mathcal{R}^{\text{true}}$. Predicted reward forms rankings and acquisitions only. During replay, measured fields replace predictions for candidates in $\mathcal{M}_t$, yielding the mixed-information score available at that step. The normalization bounds exclude held-out oracle labels. The resource term represents normalized FPGA capacity utilization and feasibility, not electrical power. MF-GARS makes no power- or energy-optimization claim. Reliable dynamic-power evaluation requires workload-dependent post-route switching activity or board-level measurements, which this benchmark does not include.

Algorithm 1 MF-GARS shared-calibration acquisition and Top-60 deployment ranking

- 1: **Input:** $\mathcal{C}_{\text{oracle}}$, $\mathcal{C}_{\text{deploy}}$, initial measurements $\mathcal{M}_0$, hardware limits, K_{acq} , $K_{\text{out}} = 60$, total calibration budget $T = 40$
 - 2: Build low-fidelity descriptors $\mathbf{x}(c)$ for both pools
 - 3: Initialize $\mathcal{M}_{|\mathcal{M}_0|} = \mathcal{M}_0$
 - 4: **for** $t = |\mathcal{M}_0|$ to $T - 1$ **do**
 - 5: Set $\mathcal{U}_t = \mathcal{C}_{\text{oracle}} \setminus \mathcal{M}_t$
 - 6: Fit bootstrap multi-task MoE surrogates using only labels in $\mathcal{M}_t$
 - 7: Predict $\hat{\Delta}(c)$, $\hat{L}(c)$, and $\hat{\mathbf{r}}(c)$ for each $c \in \mathcal{U}_t$
 - 8: Compute $\hat{A}_{\text{real}}(c)$ and $\hat{\mathcal{R}}(c)$ for all $c \in \mathcal{U}_t$
 - 9: Apply the common constraint rule and compute $\mathcal{A}(c)$ from constrained bootstrap rankings
 - 10: Select $c_t = \arg \max_{c \in \mathcal{U}_t} \mathcal{A}(c)$
 - 11: Measure c_t once to obtain $A_{\text{real}}(c_t)$, $L(c_t)$, and complete $\mathbf{r}(c_t)$; derive $\Delta(c_t)$
 - 12: Update $\mathcal{M}_{t+1} = \mathcal{M}_t \cup \{c_t\}$
 - 13: **end for**
 - 14: Freeze $\mathcal{C}_{40} = \mathcal{M}_T$ and refit the final surrogate using only $\mathcal{C}_{40}$
 - 15: Compute $I_{\text{feas}}(c)$ and $\hat{\mathcal{R}}_{\text{con}}(c)$ for every $c \in \mathcal{C}_{\text{deploy}}$
 - 16: **Return:** Top60 $_{c \in \mathcal{C}_{\text{deploy}}} \hat{\mathcal{R}}_{\text{con}}(c)$
-

Reward-greedy selection uses $\mathcal{A}_{\text{greedy}}(c) = \hat{\mathcal{R}}(c)$. Direct-Greedy differs only by modeling A_{real} directly.

We distinguish acquisition boundary size K_{acq} , evaluation shortlist size K_{eval} , and final output size K_{out} . For B bootstrap surrogates with Top- K_{acq} sets $\mathcal{T}_{K_{\text{acq}}}^{(j)}$, membership probability is

$$p_{K_{\text{acq}}}(c) = \frac{1}{B} \sum_{j=1}^B \mathbf{1}\{c \in \mathcal{T}_{K_{\text{acq}}}^{(j)}\}. \quad (23)$$

Binary entropy measures uncertainty about whether a candidate lies near the acquisition boundary:

$$\begin{aligned}H_{K_{\text{acq}}}(c) &= -p_{K_{\text{acq}}}(c) \log p_{K_{\text{acq}}}(c) \\ &\quad - (1 - p_{K_{\text{acq}}}(c)) \log(1 - p_{K_{\text{acq}}}(c)).\end{aligned}\quad (24)$$

To diversify real-CPM measurements across the coupled design inventory, $N_{s_f}(c)$ and $N_b(c)$ count measured candidates sharing the same frontend structural template and backend configuration, respectively. We define

$$Q(c) = \frac{\alpha_{s_f}}{1 + N_{s_f}(c)} + \frac{\alpha_b}{1 + N_b(c)}, \quad (25)$$

which favors under-observed frontend and backend choices. It avoids an exact-pair term, which would be constant for every unique unmeasured configuration. The acquisition is

$$\mathcal{A}(c) = p_{K_{\text{acq}}}(c) + \beta_{\text{ent}} H_{K_{\text{acq}}}(c) + \gamma_{\text{cov}} Q(c). \quad (26)$$

MF-GARS uses the retained stability-and-coverage variant. Setting $\beta_{\text{ent}} = \gamma_{\text{cov}} = 0$ gives membership-only ranking.

H. Search Algorithm

Algorithm 1 connects 40-record acquisition to the final Top-60 deployment protocol.

Here, $T = 40$ includes the initial observations in $\mathcal{M}_0$, so the loop adds exactly $40 - |\mathcal{M}_0|$ measurements. For the controlled comparison, every baseline receives the resulting $\mathcal{C}_{40}$, complete 2,680-candidate pool, and common hardware rule unchanged. Each baseline replaces only the final surrogate or ranking rule. Replay experiments reuse the loop at their stated total budgets but report $K_{\text{eval}} = 10$ instead of deployment $K_{\text{out}} = 60$. MF-GARS ranks existing configurations and does not generate architectures.

IV. EXPERIMENTS

The quantitative evaluation uses SVHN as the primary deployment benchmark and Yale Face as a cross-task face-recognition benchmark. CR and MAE denote compression ratio and mean absolute error. All experiments use $K_{\text{acq}} = 10$ for bootstrap membership and boundary uncertainty. We use $K_{\text{eval}} = 10$ for reported shortlist metrics and $K_{\text{out}} = 60$ for final constrained selection over the 2,680-candidate deployment pool.

For a held-out evaluation set, let $\mathcal{S}_{10}^{\text{pred}}$ contain the ten candidates with the highest predicted normalized deployment reward. Let $\mathcal{S}_{10}^{\text{oracle}}$ contain the ten held-out candidates with the highest true normalized deployment reward. The latter uses measured real-CPM accuracy and hardware fields. We define

$$\text{Hit@10} = 100 \frac{|\mathcal{S}_{10}^{\text{pred}} \cap \mathcal{S}_{10}^{\text{oracle}}|}{10}.$$

Hit@10 therefore measures recovery of the reward-optimal oracle shortlist, not retrieval of the highest-accuracy candidate alone. Mean@10 Reward is the average true normalized reward of the ten returned candidates. Best@10 Reward is their maximum. Replay regret is the difference between the highest eligible true reward and the best reward observed within the stated budget. The experiments below distinguish full-oracle capacity references, low-shot calibration, end-to-end replay, and conditional shared-calibration ranking by their visible records and evaluation pools.

A. Experimental Setup

All SVHN candidates share fixed training and simulation-validation splits with disjoint real-calibration and test images. RGB inputs are converted to $256 \times 256 \times 1$ grayscale frames, and classification accuracy is the primary metric. Low-shot studies resample 5/10/20/40 calibration candidates without changing the held-out test set. The physical platform couples a PCVY620 CPM frontend to a Xilinx Zynq-7020 FPGA backend. The CPM supplies real accuracy and readout statistics, while a calibrated simulator supports large-scale candidate ranking. The 320-configuration physical oracle benchmark provides the unified high-fidelity reference. Every oracle configuration has complete latency and LUT/FF/DSP/BRAM labels from the same implementation-report protocol, with no missing hardware labels. These labels characterize latency, resource capacity, and feasibility rather than power. We do

not evaluate dynamic power or energy because the benchmark lacks workload-dependent switching activity and board-level power measurements. Yale Face follows the same grayscale CPM convention and evaluates gap-aware ranking and finite-budget selection for face recognition. Exp-5 reports its quantitative results.

a) *Fixed method settings.*: We fix all coefficients and feasibility limits before forming data splits. The reward uses $\lambda_M = 0.20$, $\lambda_L = 0.10$, and $\lambda_R = 0.10$. The hardware score assigns equal weights $\omega_{\text{LUT}} = \omega_{\text{FF}} = \omega_{\text{DSP}} = \omega_{\text{BRAM}} = 0.25$. FPGA feasibility uses $B_{\text{LUT}} = 53,200$, $B_{\text{FF}} = 106,400$, $B_{\text{DSP}} = 220$, $B_{\text{BRAM}} = 140$, and $L_{\text{max}} = 1$ ms. Measured or predicted costs are divided by these limits before scoring. Coverage uses $\alpha_{sf} = \alpha_b = 0.5$. Active acquisition uses 20 bootstrap models, $\beta_{\text{ent}} = 0.2$, and $\gamma_{\text{cov}} = 0.1$. The multi-task MoE has a two-layer shared descriptor encoder with widths 128 and 64. It uses four softly routed experts with 64- and 32-unit hidden layers and linear heads for all prediction targets. Training-fold statistics standardize inputs and targets. All neural surrogates use Adam, learning rate 10^{-3} , batch size 32, at most 300 epochs, and early-stopping patience 20. These settings remain fixed across splits and budgets.

b) *Controlled baseline implementations.*: We express every external optimizer as a version-stable fixed-pool adapter over the same candidate IDs. All predictor baselines receive $\mathbf{x}(c)$, including $A_{\text{sim}}(c)$, frontend shape and bytes, $A_{\text{pure}}(b)$, and backend descriptors. Direct variants regress A_{real} , whereas gap variants regress Δ . All reward-oriented methods use the same hardware constraint and reward definition. Direct-Greedy ranks the direct regressor's predicted reward. The auto-sklearn adapter selects standardized regressors within each training fold. OpenBox-BBO and SMAC3-BBO use fixed-pool Bayesian-optimization adapters, while HyperMapper-style search applies feasibility-aware non-dominated ranking. In Figs. 9–10, all adapters receive the same frozen 40-record calibration set, 2,680 deployment candidate IDs, and constraint rule. This protocol is conditional on a calibration set selected by MF-GARS, whereas active replay gives each adapter its own queried records under the same total budget.

The hardware heads achieve R^2 values of 99.52%, 99.98%, 99.81%, 93.39%, and 99.30% for latency, LUT, FF, DSP, and BRAM, respectively (Fig. 4). These near-unit values indicate that the descriptors strongly determine most implementation quantities under the uniform HLS flow. The hardware heads are operationally used to estimate common feasibility and cost for deployment candidates without implementation reports. The result should not be interpreted as an isolated causal demonstration that auxiliary hardware supervision improves the gap head; that question requires a gap-only MoE ablation.

B. Candidate Pool and Evaluation Protocol

A candidate combines one CPM structural template, candidate-specific learned frontend parameters, and one backend configuration with its jointly learned parameters. Six structural templates and 500 FPGA backend variants form the 3,000-configuration discrete inventory. Learned parameters Θ_c distinguish candidate realizations without changing this Cartesian-product count.

TABLE I
EXTERNAL AUTOML/DSE BASELINES USED IN THE EXPERIMENTS.

Method	Configuration scope	Budget / fidelity handling	Deployment constraints	Real-CPM gap	Top- K stability
Auto-sklearn [27]	ML pipelines and hyperparameters	Bayesian optimization of software validation	No sensor/FPGA deployment constraints	✗	✗
OpenBox [29]	General black-box configurations	Finite expensive-query budget	User-defined constraints and objectives	✗	✗
SMAC3 [28]	Algorithm and hyperparameter configurations	Bayesian and generic multi-fidelity budgets	User-defined objective and cost	✗	✗
HyperMapper 2.0 [30]	Multi-objective system/accelerator configurations	Finite design-evaluation budget	FPGA feasibility and multiple objectives	✗	✗
MF-GARS (ours)	Coupled CPM-FPGA configurations	Simulation and selective real-CPM measurements	Bytes, latency, LUT/FF/DSP/BRAM	✓	✓

Prediction Quality of FPGA Hardware Metrics

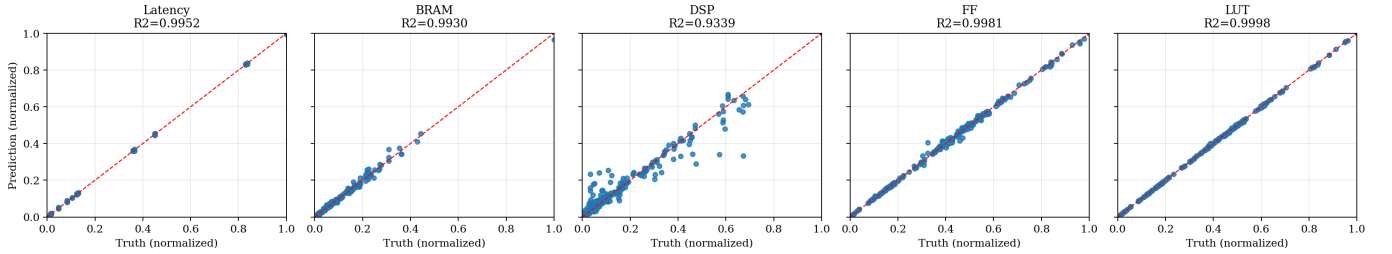


Fig. 4. Predicted versus measured latency and FPGA resource indicators. Results pool out-of-fold predictions for held-out configurations in the 320-configuration physical oracle benchmark.

The 320 IDs in $\mathcal{C}_{\text{oracle}}$ were selected and frozen before observing any real-CPM accuracy. Stratification used only pre-deployment low-fidelity information, primarily $A_{\text{pure}}(b)$ and coverage across CPM structural templates. Real-CPM accuracy was not a selection variable. Subsequent physical evaluation showed that the frozen subset spans a broad range of real-CPM accuracies. Each oracle configuration has one complete record containing real-CPM accuracy and latency/LUT/FF/DSP/BRAM labels. Because candidates are jointly trained, evaluating one ID provides a record only for that ID, even when another candidate shares its structural template. Low-measurement and active-search protocols never grant unrestricted access to all 320 records. Each experiment exposes only the records permitted by its budget, while all others remain masked for evaluation.

We separate the uses of the oracle benchmark. Fig. 4 uses candidate-level out-of-fold hardware predictions, excluding every held-out candidate from all supervised heads. Fig. 5 uses all 320 measurements only for offline correlation analysis, not surrogate training. The controlled mismatch examples in Fig. 6 also use measured oracle candidates and remain outside the final deployment screen. Table II and Fig. 7 provide full-oracle capacity references. Each random five-fold model trains on approximately 80% of the oracle benchmark, and all metrics pool predictions from unseen folds.

In each low-shot split, only 5, 10, 20, or 40 oracle records are visible for training. All remaining records stay masked for evaluation. Table III fixes the budget at 20 records and evaluates the other 300 oracle configurations. Active replay begins with five measurements and stops at total budgets 10/20/40 over 30 seeds. These budgets add 5/15/35 adaptive queries. Each method acquires records using its own policy

under the same total budget.

For final deployment ranking, MF-GARS acquires 40 visible candidate records within $\mathcal{C}_{\text{oracle}}$. The resulting $\mathcal{C}_{40}$ is frozen and shared by every method in Figs. 9–10. Each ranker uses the same records, low-fidelity descriptors, and hardware constraint. It scores all 2,680 unmeasured deployment candidates and returns a Top-60 directly from that pool. HyperMapper-style search targets constrained multi-objective coverage [30]. OpenBox-BBO optimizes fixed-pool deployment reward [29]. SMAC3-BBO uses a random-forest reward surrogate for the same task [28]. Auto-sklearn provides a generic tabular baseline [27]. Because MF-GARS acquires $\mathcal{C}_{40}$, these figures compare final rankers conditionally under identical calibration data. They do not represent all acquisition policies symmetrically. Fig. 11 and Table IV provide the separate end-to-end acquisition comparison.

Simulation accuracy correlates strongly with real-CPM accuracy (0.96), whereas compression ratio does not (0.02; Fig. 5). The measured unified hardware score $\tilde{R}_{\text{hw}}^{\text{true}}$ aligns with LUT/FF/BRAM and is anti-correlated with latency. The gap therefore refines competitive simulation candidates rather than serving as a standalone objective.

C. Exp-1: Sim-to-Real Ranking Mismatch

We first examine the ranking inconsistency that motivates gap-aware modeling. Fig. 6 reports two controlled SVHN comparisons. One varies the frontend with a fixed backend, and the other varies the backend with a fixed frontend. We repeat both comparisons on Yale Face. The cross-task tables then quantify low-shot calibration and active replay under this mismatch.

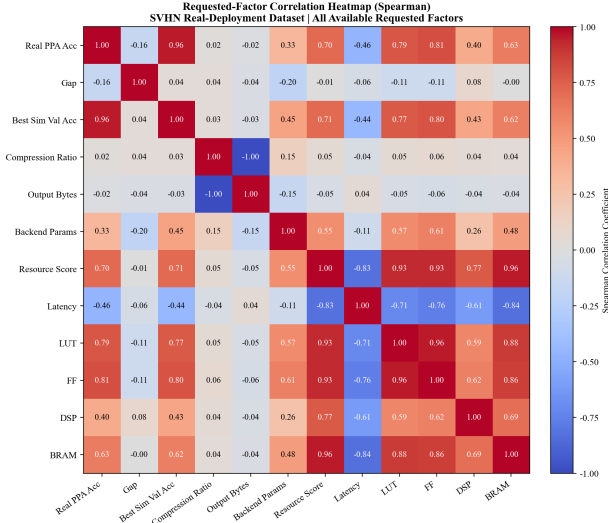


Fig. 5. Spearman correlations computed offline from all 320 physically evaluated configurations in the physical oracle benchmark; this descriptive analysis is not a surrogate-training budget.

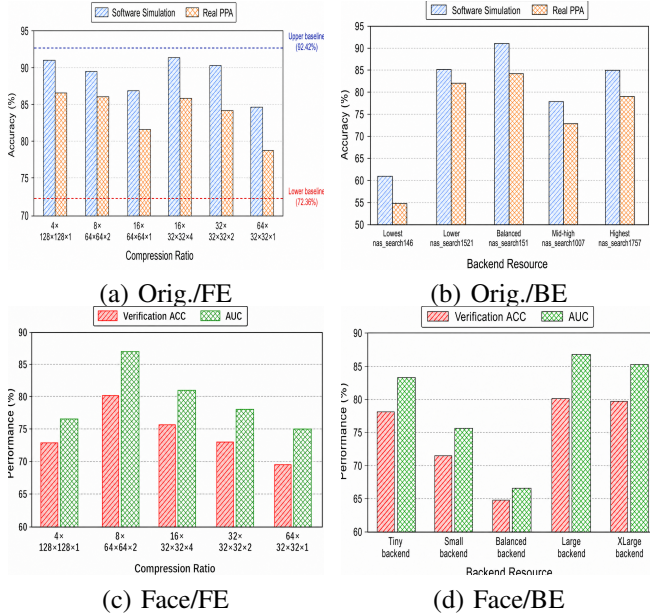


Fig. 6. Sim-to-real mismatch for SVHN (a,b) and Yale Face (c,d) with either the frontend or backend fixed.

Among candidates sharing a backend, the best simulated frontend-template pairing is not the best real-CPM candidate. Equal compression ratios can also yield different deployment accuracies (Fig. 6(a)). With a fixed frontend template, neither pure-backend nor simulated accuracy preserves the real-CPM ranking (Fig. 6(b)). Yale Face exhibits the same controlled mismatch, motivating the quantitative cross-task evaluation in Exp-5.

D. Exp-2: Low-Shot Gap Surrogate Efficiency

We evaluate candidate-level generalization on the 320-configuration physical oracle benchmark. Low-shot calibration uses 5/10/20/40 visible records. Direct- and gap-target XGBoost [32], auto-sklearn [27], multi-task MLP, and MoE

configurations compare target choice and joint model design. Fixed-pool adapters evaluate auto-sklearn, OpenBox, and SMAC3 within the same candidate space and measurement budgets as MF-GARS.

Under random five-fold evaluation, Direct-XGB obtains MAE 3.49 pp, RMSE 4.51 pp, Spearman 94.80%, and Hit@10 71%. MF-GARS obtains 2.73 pp, 3.54 pp, 97.14%, and 82%, respectively. Table II reports all predictor configurations and downstream Mean@10 true reward. Each model sees approximately 80% of the oracle configurations, with metrics pooled from out-of-fold predictions. MAE and RMSE are percentage points. Spearman, Hit@10, hardware R^2 , and normalized Mean@10 true reward are percentages. Hardware R^2 follows the order latency/LUT/FF/DSP/BRAM.

Within each predictor family in Table II, the gap-target configuration reports better prediction and ranking metrics than its direct-target counterpart. Other comparisons change both hardware-head sharing and MLP/MoE structure. They therefore compare joint configurations rather than isolating causal effects. MF-GARS gives the strongest combined full-oracle result. Relative to MoE-Direct, it reduces MAE by 0.22 pp, increases Hit@10 from 79% to 82%, and raises Mean@10 true reward from 75.37% to 76.25%.

Predictions remain near the diagonal under random, frontend-grouped, and harder backend-grouped validation (Fig. 7). This pattern indicates generalization beyond measured pairs. Low-accuracy configurations appear because the oracle benchmark is stratified across the full accuracy range. The surrogate therefore learns the complete sim-to-real relationship, not only competitive deployments. This full-oracle cross-validation is a capacity reference rather than the 40-shot deployment result. In the separate twenty-shot comparison, each of 50 shared splits exposes exactly 20 records and holds out the other 300. MAE and RMSE are percentage points. Correlation, accuracy, Hit@10, and reward are percentages, with MAE reported as mean $\pm$ standard deviation.

MF-GARS gives the lowest MAE at 5 shots (4.80 pp) and 10 shots (4.40 pp). Against Gap-XGB, paired win rates are 62% and 72%, with Wilcoxon $p = 0.0230$ and 4.65×10^{-5} , respectively (Fig. 8). At 20 shots, MF-GARS remains numerically lower (4.19 pp versus 4.32 pp), but the difference is not significant ($p = 0.0708$). At 40 shots, Gap-XGB is slightly lower (3.72 pp versus 3.74 pp). MF-GARS nevertheless gives the numerically highest Hit@10 at every budget: 70.4%, 69.0%, 68.0%, and 66.4%. Table III also shows the strongest 20-shot ranking and shortlist metrics for MF-GARS. These results support multi-task MoE-Gap as a low-sample and deployment-ranking model, not a uniformly superior pointwise regressor at every budget.

E. Controlled Shared-Calibration Deployment Ranking

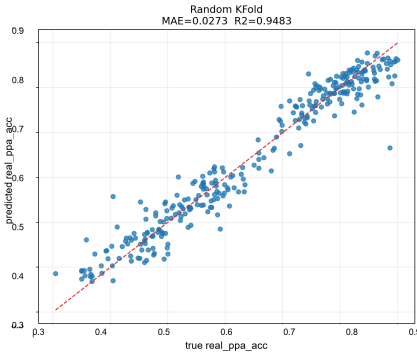
For final deployment ranking, MF-GARS acquires 40 complete real-CPM records from the oracle benchmark. The resulting calibration set is then frozen:

$$\mathcal{C}_{40} \subset \mathcal{C}_{\text{oracle}}, \quad |\mathcal{C}_{40}| = 40.$$

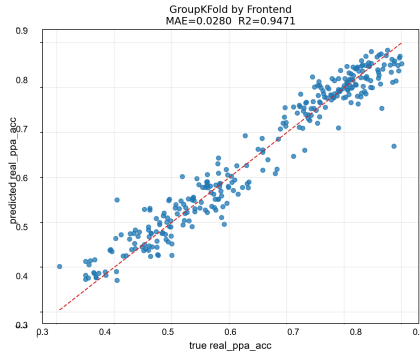
Each method trains its surrogate or ranker using the same $\mathcal{C}_{40}$ and low-fidelity candidate descriptors. Real-CPM accuracy

TABLE II
FULL-ORACLE PREDICTOR COMPARISON UNDER FIVE-FOLD CANDIDATE-LEVEL CROSS-VALIDATION.

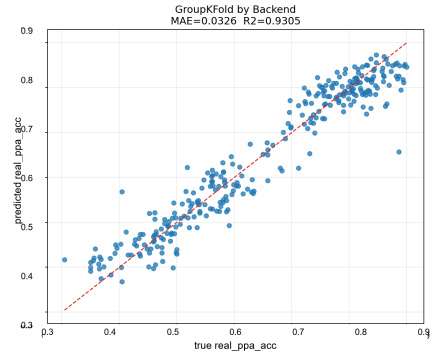
Predictor	Target	Multi-task	MoE	HW Heads	MAE (pp)	RMSE (pp)	Spearman (%)	Hit@10 (%)	HW R^2 (%; Lat./LUT/FF/DSP/BRAM)	Mean@10 True Reward (%)
Direct-XGB [32]	A_{real}	✗	✗	indep.	3.49	4.51	94.80	71	99.15 / 99.64 / 99.42 / 90.60 / 98.75	73.65
Gap-XGB [32]	Δ	✗	✗	indep.	3.18	4.17	95.68	75	99.15 / 99.64 / 99.42 / 90.60 / 98.75	74.42
Auto-sklearn-Direct [27]	A_{real}	✗	✗	indep.	3.36	4.36	95.25	73	99.23 / 99.71 / 99.50 / 91.55 / 98.84	74.03
Auto-sklearn-Gap [27]	Δ	✗	✗	indep.	2.99	3.90	96.11	78	99.23 / 99.71 / 99.50 / 91.55 / 98.84	75.11
Multi-task MLP-Direct	A_{real}	✓	✗	shared	3.11	4.02	95.89	77	99.38 / 99.82 / 99.68 / 92.60 / 99.05	74.88
Multi-task MLP-Gap	Δ	✓	✗	shared	2.82	3.67	96.59	80	99.42 / 99.87 / 99.72 / 92.95 / 99.15	75.59
MoE-Direct	A_{real}	✓	✓	shared	2.95	3.81	96.31	79	99.48 / 99.92 / 99.77 / 93.20 / 99.24	75.37
MoE-Gap (MF-GARS)	Δ	✓	✓	shared	2.73	3.54	97.14	82	99.52 / 99.98 / 99.81 / 93.39 / 99.30	76.25



(a) Random K-fold evaluation.



(b) GroupKFold by frontend.



(c) GroupKFold by backend.

Fig. 7. Full-oracle MF-GARS capacity reference under three validation protocols on the 320-configuration physical oracle benchmark. Random K-fold gives MAE = 2.73 pp and $R^2 = 94.83\%$. Grouping by frontend gives MAE = 2.80 pp and $R^2 = 94.71\%$. Grouping by backend gives MAE = 3.26 pp and $R^2 = 93.05\%$. All metrics use held-out candidate predictions.

TABLE III
TWENTY-SHOT SURROGATE AND SHORTLIST COMPARISON OVER 50 SHARED SPLITS.

Method	Target	MAE (pp)	RMSE (pp)	Spearman (%)	Top-1 Acc. (%)	Hit@10 (%)	Best@10 Acc. (%)	Mean@10 Acc. (%)	Mean@10 True Reward (%)
Direct-XGB [32]	A_{real}	5.28 ± 0.74	6.89	92.22	77.39	44.2	84.55	77.89	71.46
Gap-XGB [32]	Δ	4.32 ± 0.44	5.84	94.82	81.34	62.2	84.84	79.65	73.43
Auto-sklearn-Direct [27]	A_{real}	5.48 ± 0.86	7.20	91.31	78.49	41.2	84.51	77.76	71.07
Auto-sklearn-Gap [27]	Δ	4.53 ± 0.78	6.09	94.28	80.95	56.8	84.63	79.29	72.86
Multi-task MLP-Gap	$\Delta + \text{HW}$	5.27 ± 0.81	6.87	92.49	80.45	44.8	84.64	78.38	71.19
MF-GARS	$\Delta + \text{HW}$	4.19 ± 0.26	5.76	95.67	82.02	68.0	84.85	80.01	74.03

remains unavailable for every candidate in $\mathcal{C}_{\text{deploy}}$ during ranking. For method m , the output is

$$\mathcal{S}_{60}^{(m)} = \text{Top60}_{c \in \mathcal{C}_{\text{deploy}}} \hat{\mathcal{R}}_{\text{con},m}(c | \mathcal{C}_{40}), \quad |\mathcal{S}_{60}^{(m)}| = 60.$$

Each strategy therefore ranks the same 2,680 unmeasured candidates under identical constraints and returns 60 candidate IDs. Predicted reward determines these identities. Reported Mean@10 accuracy and reward use real-CPM measurements from the selected evaluation subset. Because MF-GARS acquired the common $\mathcal{C}_{40}$, this protocol compares final rankers conditionally under fixed calibration data. It is not an end-to-end comparison of acquisition policies.

Fig. 9 compares the measured head of each returned deployment shortlist. MF-GARS reaches Mean@10 real-CPM accuracy of 75.38%, whereas SMAC3-BBO reaches 76.00%. For the joint deployment objective, MF-GARS gives the highest Mean@10 true normalized reward at 62.35%. The

next-highest method, SMAC3-BBO, reaches 60.85%. This result supports balanced deployment ranking rather than an accuracy-only advantage. These accuracy values characterize compressed, CPM-conditioned candidates under physical read-out, not unconstrained software models evaluated on the original digital SVHN input. The intended comparison is therefore among candidates under the common accuracy-byte-latency-resource objective. Consistent with this scope, SMAC3-BBO has the higher accuracy-only Mean@10, whereas MF-GARS has the higher joint reward.

Fig. 10 separates the common candidate space from method-specific rankings. The gray background and blue reference layer use identical candidate coordinates in every panel. Between-method differences therefore reflect Top-60 and Top-1 identities, not changes to the underlying pool. The plotted coordinates are low-fidelity quantities available before deployment measurement. Each colored selection follows the

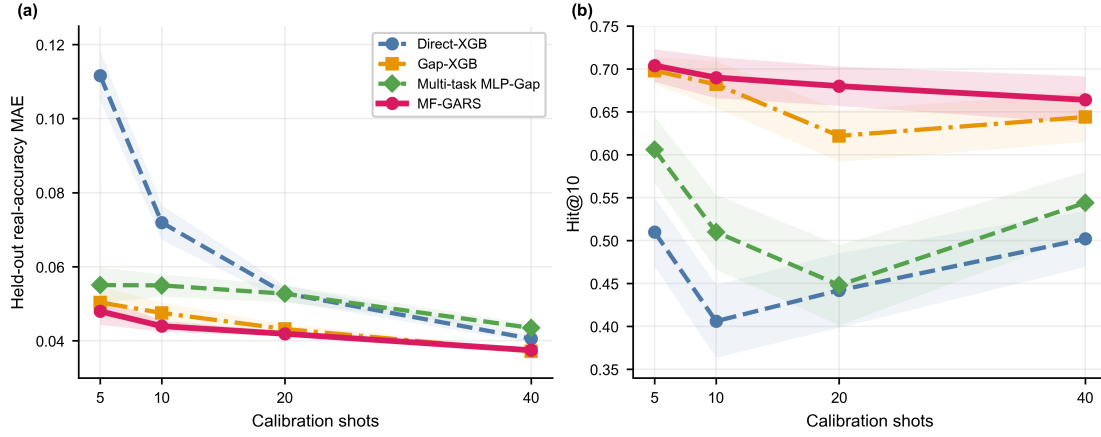


Fig. 8. Low-shot sim-to-real calibration over 50 shared splits. Training exposes 5, 10, 20, or 40 oracle records, while all others remain masked. (a) Held-out real-CPM accuracy MAE in percentage points. (b) Hit@10 in percent, defined as the fraction of the true-reward oracle Top-10 recovered by the predicted-reward Top-10. Curves show means, and shaded regions show two-sided 95% confidence intervals. Lower is better in (a); higher is better in (b).

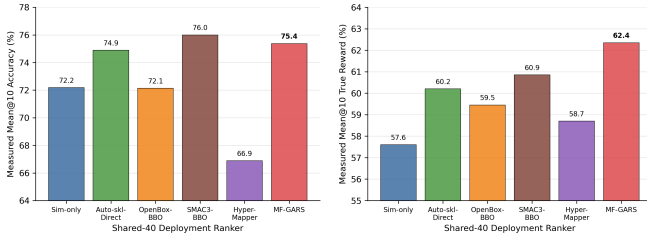


Fig. 9. Shared- C_{40} deployment-ranking quality over the 2,680-candidate pool.

corresponding predicted, constraint-aware reward. True reward appears only in the post-selection evaluation in Fig. 9.

F. Exp-3: Top-K Stability Active Screening Replay

Sequential replay treats the 320-configuration physical oracle benchmark as a controlled masked-record environment. All complete records exist offline, but replay makes a record visible only after querying its candidate. Each measurement obtains real-CPM accuracy, latency, and the complete LUT/FF/DSP/BRAM vector. It provides no field for unqueried candidates. Every method starts from the same five measurements and queries one hidden record per step. The measured-set size reaches total budgets 10/20/40 over 30 seeds ($K_{\text{acq}} = K_{\text{eval}} = 10$, 20 bootstraps, $\beta_{\text{ent}} = 0.2$, $\gamma_{\text{cov}} = 0.1$). These settings contain 5/15/35 adaptive queries after initialization. At each budget, the final deployment Top-10 includes measured candidates and the predicted unmeasured shortlist. Strong measured candidates therefore remain eligible for deployment. Unlike Figs. 9–10, this end-to-end replay allows each method to follow its own acquisition policy under the same budget. Table IV reports normalized rewards and accuracy as percentages. Regret and paired gain are percentage-point differences, with gain measured against Direct-Greedy deployment Mean@10 true reward.

At total budget 20, five initial and 15 adaptive measurements, MF-GARS reaches Best@10 reward 71.81%. This value ties Direct-Greedy, Auto-sklearn-Direct, and HyperMap-

per under the inclusive deployment metric (Fig. 11a, Table IV). The metric does not reward leaving the best candidate unmeasured. MF-GARS gives the highest deployment Mean@10 reward, 61.45%, a 3.24-point gain over Direct-Greedy. Its best-observed reward remains 70.62%. The comparison therefore distinguishes measurement discovery from shortlist quality while retaining measured candidates for deployment.

Zero standard deviation in a best-endpoint metric means every shared seed includes the same unique reward-optimal candidate in its inclusive deployment Top-10. Other shortlist members may still differ across seeds, as shown by variation in Mean@10 reward and aggregate statistics. The zero values therefore indicate a stable optimum rather than identical shortlists.

G. Exp-4: AutoML/DSE Baselines and Multi-objective Trade-offs

Shortlist quality and broad Pareto coverage are distinct objectives. Fig. 11b evaluates coverage across measured accuracy, bytes, measured latency, and measured unified FPGA resource score $\hat{R}_{\text{hw}}^{\text{true}}$.

At budget 10, MF-GARS obtains normalized hypervolume 57.43%, compared with 53.41% for HyperMapper-style search. The paired win rate is 70%, with $p = 0.00814$. MF-GARS also exceeds Direct-Greedy by 6.92 percentage points ($p = 1.68 \times 10^{-6}$). HyperMapper overtakes MF-GARS at budget 20 (62.89% versus 60.01%) and budget 40 (66.25% versus 61.61%). MF-GARS therefore identifies useful trade-offs earlier, whereas HyperMapper provides stronger high-budget Pareto coverage.

H. Exp-5: Yale Face Cross-Task Validation

The Yale Face Database [10] provides a cross-task face-recognition distribution of 165 grayscale images from 15 individuals. We retain the SVHN grayscale CPM convention, reward normalization, and finite-budget ranking protocol. Evaluation includes twenty-shot surrogate quality over 50

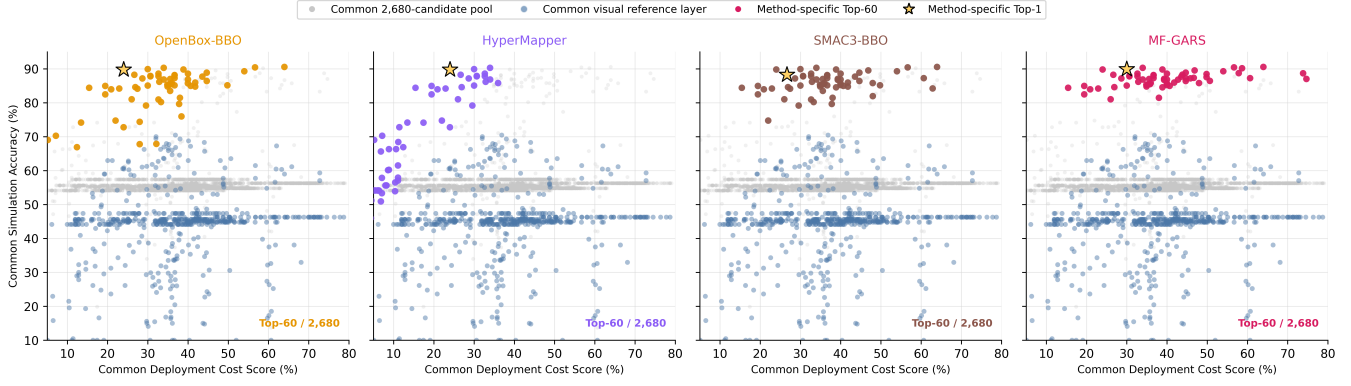
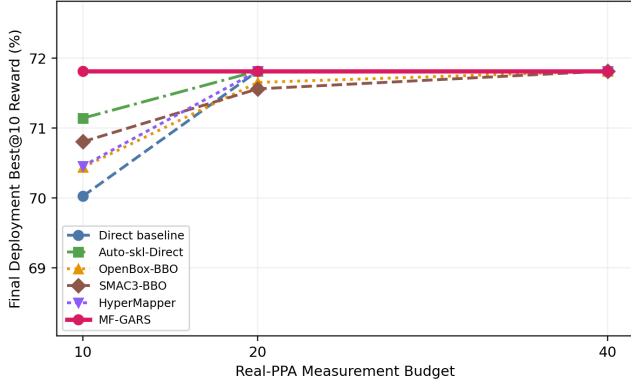


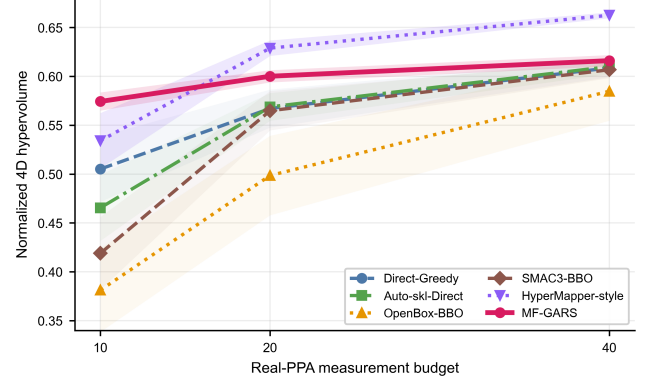
Fig. 10. Common-coordinate Top-60 selection from the 2,680-candidate deployment pool. Every panel repeats the same candidate IDs at identical method-independent coordinates: deployment cost horizontally and simulation accuracy vertically. Blue points form a fixed density-preserving reference layer within the common background, not a reduced search space or eligibility set. Each ranker scores all 2,680 candidates under the common hardware constraint. Colored markers show the method-specific Top-60, and the star shows its Top-1.

TABLE IV
BUDGET-20 END-TO-END ACTIVE REPLAY OVER 30 SHARED SEEDS.

Method	Best Obs. True Reward (%)	Regret (pp)	Deployment Best@10 True Reward (%)	Deployment Mean@10 True Reward (%)	Deployment Best@10 Acc. (%)	Bytes@10	Paired Gain (pp)
Direct-Greedy [32]	71.81 $\pm$ 0.00	0.00 $\pm$ 0.00	71.81 $\pm$ 0.00	58.21 $\pm$ 1.95	85.00 $\pm$ 0.00	2700.8	+0.00
Auto-sklearn-Direct-Greedy [27]	71.81 $\pm$ 0.00	0.00 $\pm$ 0.00	71.81 $\pm$ 0.00	57.86 $\pm$ 1.32	85.00 $\pm$ 0.00	2291.2	-0.35
OpenBox-BBO [29]	71.65 $\pm$ 0.30	0.16 $\pm$ 0.30	71.65 $\pm$ 0.30	57.91 $\pm$ 5.80	84.86 $\pm$ 0.26	2304.0	-0.30
SMAC3-BBO [28]	71.56 $\pm$ 0.72	0.26 $\pm$ 0.72	71.56 $\pm$ 0.72	58.42 $\pm$ 2.90	84.50 $\pm$ 0.20	2726.4	+0.21
HyperMapper-style [30]	71.81 $\pm$ 0.00	0.00 $\pm$ 0.00	71.81 $\pm$ 0.00	53.48 $\pm$ 2.75	85.00 $\pm$ 0.00	1843.2	-4.73
MF-GARS	70.62 $\pm$ 1.53	1.19 $\pm$ 1.53	71.81 $\pm$ 0.00	61.45 $\pm$ 1.45	85.00 $\pm$ 0.00	2176.0	+3.24



(a) Final deployment Best@10 true reward.



(b) Normalized four-dimensional hypervolume.

Fig. 11. Finite-budget masked-record replay and multi-objective coverage on the 320-configuration physical oracle benchmark over 30 shared initial sets. Budgets 10/20/40 include the five initial real-CPM measurements. Each method follows its own acquisition policy, and only queried configurations make complete records available. (a) Inclusive final deployment Top-10 quality, with measured candidates remaining eligible. (b) Hypervolume maximizing measured real-CPM accuracy while minimizing bytes, measured latency, and measured unified FPGA resource score R_{hw}^{true} . Curves show means, and shaded regions show two-sided 95% confidence intervals.

shared splits and budget-20 active replay over 30 shared seeds. In Table V, MAE and RMSE are percentage points. Spearman, Hit@10, accuracy, and normalized Mean@10 true reward are percentages.

The twenty-shot results show that gap-target learning transfers to Yale Face. Relative to Direct-XGB, MF-GARS reduces MAE from 5.10 to 3.76 pp and RMSE from 6.42 to 4.89 pp. Spearman correlation rises from 89.40% to 95.20%, and Hit@10 rises from 42.0% to 70.0%. Gap-XGB is the strongest non-MoE predictor in this table. MF-GARS still lowers its MAE by 0.52 pp and raises Hit@10 by 12.0 percentage points. Mean@10 accuracy and reward improve by 1.47 and

1.85 points, respectively. The joint improvement suggests that shared hardware supervision and expert routing retain useful ranking information under different face-recognition statistics. For Table VI, the total budget of 20 includes the five initial measurements. Normalized reward and accuracy are percentages. Regret and paired gain are percentage-point differences over 30 shared seeds.

At total measurement budget 20, MF-GARS obtains Yale deployment Mean@10 reward of 61.02%, a 2.85-point gain over Direct-Greedy (58.17%). Its Best@10 reward is 71.39%, tied with HyperMapper-style search, while its 2,048-byte shortlist is smaller than Direct-Greedy's 2,688 bytes. Hy-

TABLE V
YALE FACE TWENTY-SHOT SURROGATE AND SHORTLIST RESULTS.

Method	MAE (pp)	RMSE (pp)	Spearman (%)	Hit@10 (%)	Best@10 Acc. (%)	Mean@10 Acc. (%)	Mean@10 True Reward (%)
Direct-XGB [32]	5.10 $\pm$ 0.68	6.42	89.40	42.0	92.10	86.40	72.35
Gap-XGB [32]	4.28 $\pm$ 0.51	5.55	92.60	58.0	93.20	88.15	74.20
Auto-sklearn-Gap [27]	4.43 $\pm$ 0.57	5.70	91.90	54.0	93.05	87.80	73.90
Multi-task MLP-Gap	4.65 $\pm$ 0.62	5.94	91.40	51.0	92.80	87.10	73.10
MF-GARS	3.76 $\pm$ 0.34	4.89	95.20	70.0	94.25	89.62	76.05

TABLE VI
YALE FACE BUDGET-20 ACTIVE-REPLAY RESULTS.

Method	Best Obs. True Reward (%)	Regret (pp)	Deployment Best@10 True Reward (%)	Deployment Mean@10 True Reward (%)	Deployment Best@10 Acc. (%)	Bytes@10	Paired Gain (pp)
Direct-Greedy [32]	70.84 $\pm$ 0.80	0.55 $\pm$ 0.80	70.84 $\pm$ 0.80	58.17 $\pm$ 1.72	93.80 $\pm$ 0.42	2688.0	+0.00
Auto-sklearn-Direct-Greedy [27]	70.36 $\pm$ 1.05	1.03 $\pm$ 1.05	70.75 $\pm$ 0.92	57.70 $\pm$ 1.64	93.65 $\pm$ 0.50	2368.0	-0.47
OpenBox-BBO [29]	70.10 $\pm$ 1.22	1.29 $\pm$ 1.22	70.40 $\pm$ 1.10	56.92 $\pm$ 2.28	93.20 $\pm$ 0.56	2304.0	-1.25
SMAC3-BBO [28]	69.95 $\pm$ 1.34	1.44 $\pm$ 1.34	70.20 $\pm$ 1.15	57.25 $\pm$ 2.06	93.00 $\pm$ 0.62	2560.0	-0.92
HyperMapper-style [30]	71.39 $\pm$ 0.00	0.00 $\pm$ 0.00	71.39 $\pm$ 0.00	56.80 $\pm$ 2.50	94.25 $\pm$ 0.00	1792.0	-1.37
MF-GARS	70.58 $\pm$ 1.10	0.81 $\pm$ 1.10	71.39 $\pm$ 0.00	61.02 $\pm$ 1.38	94.25 $\pm$ 0.00	2048.0	+2.85

perMapper remains better on bytes, so the result reflects a shortlist-quality advantage rather than uniform dominance on every dimension.

I. Component Comparison and Boundary Check

Table II shows empirical gains from gap targets within matched predictor families. The multi-task MoE-Gap configuration gives the strongest combined result. Some rows change several modeling choices simultaneously. The table therefore supports component-level comparison rather than fully orthogonal causal attribution. Fig. 8 shows that the point-wise MAE advantage is strongest with scarce real records. This result should be interpreted alongside the more stable Top- K retrieval trend. Replay establishes the complementary boundary: MF-GARS improves finite-budget shortlist quality, whereas HyperMapper is preferable for high-budget Pareto coverage.

V. DISCUSSION

MF-GARS treats CPM-FPGA deployment as a coupled physical configuration problem. Gap modeling preserves simulation information while calibrating rankings from limited measurements, and Top- K stability, boundary uncertainty, and coverage guide finite-budget screening. Paired Wilcoxon tests support the MAE advantage at 5 and 10 shots, but not at 20 shots; Gap-XGB is slightly better at 40 shots. MF-GARS nevertheless retains the numerically highest Hit@10 across budgets. HyperMapper-style search remains preferable for broad high-budget Pareto coverage, and the shared- C_{40} result is conditional on MF-GARS-selected calibration data rather than a symmetric acquisition comparison.

The experiments evaluate per-frame classification on SVHN and Yale Face. Temporal consistency, motion-aware inference, sustained video frame rate, and workload-dependent power remain unverified. Future work should test temporal workloads, binding hardware budgets, workload-grounded power, additional sensors and FPGA targets, and searchable CPM-FPGA partitions.

VI. CONCLUSION

MF-GARS formulates heterogeneous CPM-FPGA deployment as multi-fidelity AutoML over jointly trained candidates. Its MoE-Gap surrogate calibrates simulation-based rankings and predicts FPGA costs, while its Top- K policy allocates limited measurements. MAE reductions are supported at 5 and 10 shots, not at 20 shots; Gap-XGB is slightly better at 40 shots, whereas MF-GARS has the numerically highest Hit@10 at every budget. Budget-20 replay improves true deployment Mean@10 reward on both SVHN and Yale Face. The 320 physical records form a frozen oracle benchmark, and a separate conditional study ranks all 2,680 deployment candidates from one shared 40-record calibration set. Current evidence is limited to per-frame CPM-FPGA ranking; temporal video behavior and power optimization remain outside the validated scope.

REFERENCES

- [1] Y. Liu, L. N. Bose, J. Chen, S. J. Carey, P. Dudek, and W. W. Mayol-Cuevas, "High-speed light-weight CNN inference via strided convolutions on a pixel processor array," in *British Machine Vision Conference (BMVC)*, 2020.
- [2] Y. Liu and Y. Lu, "On-sensor binarized fully convolutional neural network for localisation and coarse segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 3629–3638.
- [3] Y. Liu, L. Bose, R. Fan, P. Dudek, and W. Mayol-Cuevas, "On-sensor binarized CNN inference with dynamic model swapping in pixel processor arrays," *Frontiers in Neuroscience*, vol. 16, p. 909448, 2022.
- [4] Y. Liu, R. Fan, J. Guo, H. Ni, and M. U. M. Bhutta, "In-sensor visual perception and inference," *Intelligent Computing*, vol. 2, 2023, art. no. 0043.
- [5] P. Dudek, T. Richardson, L. Bose, S. Carey, J. Chen, C. Greatwood, Y. Liu, and W. Mayol-Cuevas, "Sensor-level computer vision with pixel processor arrays for agile robots," *Science Robotics*, vol. 7, no. 67, p. eabl7755, 2022.
- [6] D. T. Nguyen, H. Kim, and H.-J. Lee, "Layer-specific optimization for mixed data flow with mixed precision in FPGA design for CNN-based object detectors," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 6, pp. 2450–2464, 2020.
- [7] J.-W. Chang, K.-W. Kang, and S.-J. Kang, "An energy-efficient FPGA-based deconvolutional neural networks accelerator for single image super-resolution," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 1, pp. 281–295, 2018.
- [8] S. Li, Z. Zhu, H. Sun, X. Ning, G. Dai, Y. Hu, H. Yang, and Y. Wang, "Toward high-accuracy and real-time two-stage small object detection on FPGA," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 9, pp. 8053–8066, 2024.
- [9] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, Granada, Spain, 2011.
- [10] P. N. Belhumeur, J. P. Hespanha, and D. J. Kriegman, "Eigenfaces vs. fisherfaces: Recognition using class specific linear projection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 19, no. 7, pp. 711–720, 1997.

- [11] S. J. Carey, A. Lopich, D. R. Barr, B. Wang, and P. Dudek, "A 100,000 fps vision sensor with embedded 535 GOPS/W 256×256 SIMD processor array," in *2013 Symposium on VLSI Circuits*. IEEE, 2013, pp. C182–C183.
- [12] L. Bose, J. Chen, S. J. Carey, P. Dudek, and W. Mayol-Cuevas, "A camera that CNNs: Towards embedded neural networks on pixel processor arrays," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1335–1344.
- [13] L. Bose, P. Dudek, J. Chen, S. J. Carey, and W. W. Mayol-Cuevas, "Fully embedding fast convolutional networks on pixel processor arrays," in *European Conference on Computer Vision*. Springer, 2020, pp. 488–503.
- [14] H. M. So, L. Bose, P. Dudek, and G. Wetzstein, "Pixelrnn: in-pixel recurrent neural networks for end-to-end-optimized perception with neural sensors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 25 233–25 244.
- [15] C. Jia, X. Hang, S. Wang, Y. Wu, S. Ma, and W. Gao, "FPX-NIC: An FPGA-accelerated 4K ultra-high-definition neural video coding system," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 9, pp. 6385–6399, 2022.
- [16] X. Zhang, Y. Ma, J. Xiong, W.-M. W. Hwu, V. Kindratenko, and D. Chen, "Exploring HW/SW co-design for video analysis on CPU-FPGA heterogeneous systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 6, pp. 1606–1619, 2021.
- [17] S. Sun, T. He, and Z. Chen, "Semantic structured image coding framework for multiple intelligent applications," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 9, pp. 3631–3642, 2020.
- [18] R. Torfason, F. Mentzer, E. Agustsson, M. Tschannen, R. Timofte, and L. Van Gool, "Towards image understanding from deep compression without decoding," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, Vancouver, BC, Canada, 2018. [Online]. Available: <https://openreview.net/forum?id=HkXWCMbRW>
- [19] Y. Hu, S. Yang, W. Yang, L.-Y. Duan, and J. Liu, "Towards coding for human and machine vision: A scalable image coding approach," in *2020 IEEE International Conference on Multimedia and Expo (ICME)*, London, UK, 2020, pp. 1–6.
- [20] X. Li, J. Shi, and Z. Chen, "Task-driven semantic coding via reinforcement learning," *IEEE Transactions on Image Processing*, vol. 30, pp. 6307–6320, 2021.
- [21] N. Yan, C. Gao, D. Liu, H. Li, L. Li, and F. Wu, "Sssic: Semantics-to-signal scalable image coding with learned structural representations," *IEEE Transactions on Image Processing*, vol. 30, pp. 8939–8954, 2021.
- [22] Y. Kim, H. Jeong, J. Yu, Y. Kim, J. Lee, S. Y. Jeong, and H. Y. Kim, "End-to-end learnable multi-scale feature compression for VCM," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 5, pp. 3156–3167, 2023.
- [23] P. Zhang, S. Wang, M. Wang, J. Li, X. Wang, and S. Kwong, "Rethinking semantic image compression: Scalable representation with cross-modality transfer," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 33, no. 8, pp. 4441–4445, 2023.
- [24] G. Lu, X. Ge, T. Zhong, Q. Hu, and J. Geng, "Preprocessing enhanced image compression for machine vision," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 12, pp. 13 556–13 568, 2024.
- [25] Y. Fu, Y. Zhang, Y. Wang, Z. Lu, V. Boominathan, A. Veeraraghavan, and Y. Lin, "SACoD: Sensor algorithm co-design towards efficient CNN-powered intelligent PhlatCam," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 5168–5177.
- [26] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013, pp. 847–855.
- [27] M. Feurer, A. Klein, K. Eggersperger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015. [Online]. Available: <https://proceedings.neurips.cc/paper/2015/hash/11d0e6287202fced83f79975ec59a3a6-Abstract.html>
- [28] M. Lindauer, K. Eggersperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "SMAC3: A versatile bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022. [Online]. Available: <https://www.jmlr.org/papers/v23/21-0888.html>
- [29] Y. Li, Y. Shen, W. Zhang, Y. Chen, H. Jiang, M. Liu, J. Jiang, J. Gao, W. Wu, Z. Yang, C. Zhang, and B. Cui, "OpenBox: A generalized black-box optimization service," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2021, pp. 3209–3219.
- [30] L. Nardi, D. Koepfinger, and K. Olukotun, "Practical design space exploration," in *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2019, pp. 347–358.
- [31] A. Sohrabzadeh, C. H. Yu, M. Gao, and J. Cong, "AutoDSE: Enabling software programmers to design efficient FPGA accelerators," *ACM Transactions on Design Automation of Electronic Systems*, vol. 27, no. 4, pp. 1–27, 2022.
- [32] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.